P3

Sustracción

Sustraccion1 — T(n) = T(n-1) + O(1)
La función rec1 realiza un trabajo O(1) en cada nivel (cont++) y realiza una sola llamada recursiva con n-1. Aplicando el Teorema de la Sustracción con a=1, b=1, k=0:
T(n) = T(n-1) + O(n^0) = O(n^1) = O(n)
Los tiempos crecen linealmente con n, lo que coincide con la complejidad O(n).

Sustraccion2 — Recurrencia: T(n) = T(n-1) + O(n).
La función rec2 realiza dos bucles de n iteraciones cada uno (2·O(n) = O(n)) y una sola llamada recursiva. Con a=1, b=1, k=1:
T(n) = T(n-1) + O(n^1) = O(n^2)
Los tiempos deben crecer cuadráticamente.

Sustraccion3 — Recurrencia: T(n) = 2·T(n-1) + O(1).
La función rec3 realiza trabajo O(1) y realiza dos llamadas recursivas con n-1. Con a=2, b=1, k=0:
T(n) = 2·T(n-1) + O(1) = O(2^n)
El tiempo de ejecución crece exponencialmente con n.

- ¿Para qué valor de n dejan de dar tiempo (abortan) las clases Sustracción1 y Sustracción2? ¿Por qué sucede eso?
  Para n = 5.000 abortan ya que la pila se llena.

- ¿Cuántos años tardaría en finalizar la ejecución Sustracción3 para n = 80? Razonar la respuesta.
  Sustraccion3 tiene complejidad O(2^n). Para n = 80: T(80) = 2^80 operaciones
  Hacer regla de 3 con lo que tarda para otra n.
  Tomando como referencia que para n = 40 el tiempo es de 1 segundo, calculamos:

$$\frac{T(80)}{T(40)} = \frac{2^{80}}{2^{40}} = 2^{40} = 1{,}1 \times 10^{12}\ s$$

  Si T(40) = 1 s, entonces T(80) = $1{,}1 \times 10^{12}$ s, que pasados a años serían 34.000.

TABLA SUSTRACCIÓN4

n	tiempo
100	1
200	8
400	63
800	511
1600	4082
3200	32654
6400	FdT

TABLA SUSTRACCIÓN5

n	tiempo
30	2
32	5
34	16
36	47
38	142
40	428
42	1283
44	3851
46	11553
48	34660
50	FdT

- ¿Cuántos años tardaría en finalizar la ejecución Sustracción5 para n = 80? Razonar la respuesta.
  La clase Sustracción5 tiene una complejidad de $O(3^{n/2})$.
  Entonces sabemos que para n = 80: $3^{80/2} = 1{,}2 \times 10^{19}$ operaciones.
  Tomando como referencia T(30) = 1 ms:

$$\frac{T(80)}{T(30)} = \frac{3^{40}}{3^{15}} = 3^{25} = 8{,}5 \times 10^{11}\ ms = 27\ años$$

División

Division1 — T(n) = T(n/3) + O(n)
La función rec1 tiene un bucle O(n) (y una sola llamada recursiva con n/3. Con a=1, b=3, k=1: a < b^k (1 < 3):
T(n) = O(n^k) = O(n)
Los tiempos crecen linealmente. Confirmado: al doblar n el tiempo se dobla.

Division2 — T(n) = 2·T(n/2) + O(n)
La función rec2 tiene un bucle O(n) y dos llamadas recursivas con n/2. Con a=2, b=2, k=1: a = b^k (2 = 2):
T(n) = O(n^k · log n) = O(nlog n)
Los tiempos crecen como nlog(n).

Division3 — T(n) = 2·T(n/2) + O(1)
La función rec3 tiene trabajo O(1) (cont++) y dos llamadas recursivas con n/2. Con a=2, b=2, k=0: a > b^k (2 > 1):
T(n) = O(n^log_b(a)) = O(n^log_2(2)) = O(n)
Los tiempos crecen linealmente.

TABLA DIVISION4

n	tiempo
1000	3
2000	12
4000	49
8000	196
16000	783
32000	3135
64000	12541
128000	50163
256000	FdT

TABLA DIVISION5

n	tiempo
1000	18
2000	71
4000	285
8000	1141
16000	4563
32000	18253
64000	52917
128000	FdT

TABLA VECTORSUMA

	VectorSuma			
n	40	100	200	Complejidad
Metodo1	229	505	893	O(n)
Metodo2	598	1466	2919	O(n)
Metodo3	1415	3560	7091	O(nlog(n))

TABLA FIBONACCI

	Fibonacci	
n	40	Complejidad
Metodo1	229	O(n)
Metodo2	598	O(n)
Metodo3	1415	O(n)
Metodo4	4345	O(1.6^n)

DyV

TABLA PUNTOSTRIVIAL

n	tiempo
1024	5
2048	7
4096	21
8192	54
16384	218
32768	797
65536	3322
131072	12902
262144	57864
524288	FdT

PUNTOS DyV

n	tiempo
1024	2
2048	4
4096	9
8192	20
16384	45
32768	100
65536	220
131072	467
262144	1010
524288	2223
1048576	5007
2097152	FdT